Object Oriented Programming Lab (CSEG1044)
Experiment-11&12: Exception Handling + Multithreading
(Student Handout — Before Submission)

Tofik Ali

March 3, 2026

Contents

1 About this handout **1**

2 Experiment 11 (Syllabus) **1**

3 Experiment 12 (Syllabus) **1**

4 What you must do (minimum requirements) **2**
4.1 Task A — Exception handling (Exp-11) 2
4.2 Task B — Multithreading + synchronization (Exp-12) 2

5 Hints (read only if stuck) **2**

6 What to submit (MANDATORY) **3**

1 About this handout

This handout describes the required programs for exceptions and multithreading. It provides requirements and common pitfalls, but not full solutions.

2 Experiment 11 (Syllabus)

- **Objective:** To handle runtime errors using exception handling mechanisms.
- **Problem Statement:** Develop programs using try-catch-finally blocks.
- **Expected Outcome:** Students will write robust programs.

3 Experiment 12 (Syllabus)

- **Objective:** To implement multithreading in Java.
- **Problem Statement:** Create threads using `Thread` class and `Runnable` interface.

- **Expected Outcome:** Students will understand concurrent programming.

4 What you must do (minimum requirements)

4.1 Task A — Exception handling (Exp-11)

Create `ExceptionHandlingDemo.java`:

1. Read integers `a` and `b`, compute `a / b`.
2. Handle:
   - divide-by-zero (`ArithmeticException`)
   - invalid numeric input (scanner mismatch or parse error)
3. Use `finally` to print `Done` (and close resources if any).
4. Create a custom checked exception `InvalidMarksException extends Exception`.
5. Write `validateMarks(int m) throws InvalidMarksException` and throw it for marks not in `0..100`.
6. Catch and print a friendly message.

4.2 Task B — Multithreading + synchronization (Exp-12)

Create `ThreadSyncDemo.java`:

1. Create a shared `Counter` with:
   - `incUnsafe()` (no synchronization)
   - `incSafe()` (use `synchronized`)
2. Create two threads that increment the same counter many times (example: 100000 each).
3. Use `start()` and `join()`.
4. Print final counter values for unsafe and safe runs.
5. Write a short explanation (2–4 lines): why unsafe output differs from expected.

5 Hints (read only if stuck)

- `finally` executes whether an exception occurs or not.
- Checked exceptions must be handled or declared using `throws`.
- Calling `run()` does not start a new thread; call `start()`.
- `v++` is not atomic; `synchronized` protects the critical section.

6 What to submit (MANDATORY)

Submit a single zipped folder containing:

- `ExceptionHandlingDemo.java`
- `ThreadSyncDemo.java`
- sample output for both programs
- observations (5–10 lines): try/catch/finally order; what race condition means

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed March 3, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed March 3, 2026).